task2

September 27, 2026

[16]:
```
import numpy as np
import pandas as pd

def calculate_entropy(y):
    _, counts = np.unique(y, return_counts=True)
    probabilities = counts / len(y)

    probabilities = probabilities[probabilities > 0]
    return -np.sum(probabilities * np.log2(probabilities))

def calculate_information_gain(y, y_left, y_right):
    weight_left = len(y_left) / len(y)
    weight_right = len(y_right) / len(y)

    entropy_parent = calculate_entropy(y)
    entropy_children = (weight_left * calculate_entropy(y_left)) +␣
 ↪(weight_right * calculate_entropy(y_right))

    return entropy_parent - entropy_children
```

[ ]:
```
class Node:
    """Caches numerical split criteria and tree structure."""
    def __init__(self, feature=None, threshold=None, left=None, right=None, *,␣
 ↪value=None):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value

    def is_leaf_node(self):
        return self.value is not None


class DecisionTree:
    def __init__(self, min_samples_split=2, max_depth=4):
        self.min_samples_split = min_samples_split
```

```
        self.max_depth = max_depth
        self.root = None

    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y)
        self.root = self._build_tree(X, y, depth=0)

    def _build_tree(self, X, y, depth):
        n_samples, n_features = X.shape
        n_labels = len(np.unique(y))

        if (depth >= self.max_depth or n_samples < self.min_samples_split or␣
↪n_labels == 1):
            leaf_value = self._most_common_label(y)
            return Node(value=leaf_value)

        best_feat, best_thresh = self._get_best_split(X, y)

        if best_feat is None:
            leaf_value = self._most_common_label(y)
            return Node(value=leaf_value)

        left_idxs = X[:, best_feat] <= best_thresh
        right_idxs = ~left_idxs

        left_child = self._build_tree(X[left_idxs], y[left_idxs], depth + 1)
        right_child = self._build_tree(X[right_idxs], y[right_idxs], depth + 1)

        return Node(best_feat, best_thresh, left_child, right_child)

    def _get_best_split(self, X, y):
        best_gain = -1
        split_idx, split_threshold = None, None
        _, n_features = X.shape

        for feature_idx in range(n_features):
            column_data = X[:, feature_idx]
            unique_values = np.sort(np.unique(column_data))
            midpoints = (unique_values[:-1] + unique_values[1:]) / 2

            for threshold in midpoints:
                left_idxs = column_data <= threshold
                right_idxs = column_data > threshold

                if len(y[left_idxs]) == 0 or len(y[right_idxs]) == 0:
                    continue
```

```
                gain = calculate_information_gain(y, y[left_idxs],␣
↪y[right_idxs])

                if gain > best_gain:
                    best_gain = gain
                    split_idx = feature_idx
                    split_threshold = threshold

        return split_idx, split_threshold

    def _most_common_label(self, y):
        values, counts = np.unique(y, return_counts=True)
        return values[np.argmax(counts)]

    def predict(self, X):
        X = np.array(X)
        return np.array([self._traverse_tree(x, self.root) for x in X])

    def _traverse_tree(self, x, node):
        if node.is_leaf_node():
            return node.value

        if x[node.feature] <= node.threshold:
            return self._traverse_tree(x, node.left)
        return self._traverse_tree(x, node.right)
```

[18]:
```
df = pd.read_csv("AAPL.csv")
df.columns = df.columns.str.lower()
df["close_pct_change"] = df["adj_close"].pct_change()
df["vol_pct_change"] = df["adj_volume"].pct_change()
df["target_direction"] = (df["adj_close"].shift(-1) > df["adj_close"]).
↪astype(int)
df = df.dropna().reset_index(drop=True)
feature_columns = ["adj_close", "adj_volume", "close_pct_change",␣
↪"vol_pct_change"]
X = df[feature_columns]
y = df["target_direction"]

split_index = int(len(X) * 0.80)

X_train, y_train = X.iloc[:split_index], y.iloc[:split_index]
X_test, y_test = X.iloc[split_index:], y.iloc[split_index:]

tree = DecisionTree(max_depth=4, min_samples_split=2)
tree.fit(X_train, y_train)
```

```python
predictions = tree.predict(X_test)

correct_predictions = np.sum(np.array(y_test) == predictions)
accuracy = correct_predictions / len(y_test)

print(f"Train Rows: {len(X_train)} | Test Rows: {len(X_test)}")
print(f"Directional Prediction Accuracy: {accuracy * 100:.2f}%")
```

```
Train Rows: 1273 | Test Rows: 319
Directional Prediction Accuracy: 46.71%
```